Systems programming lesson 5 & 6 combined

COM 312: BSc Computer science

Topics area coverage.

UNIX inter process communication Tools

- Topic introduction
- Pipes and practices using P open () and P close () functions

UNIX Thread management and synchronization

- Posix standards and its sub group
- Threading Concept in systems programming
- Thread ID calls
- Mutex and semaphore in systems administration

UNIX inter process communication Tool

Defination1: - Inter process communication is the mechanism where process communicate with each other and with the kernel to coordinate their activities.to synchronize their action.

Definition 2: - it is a set of programming interface (API) which allows a programmer to coordinate activities among various program processes which can run concurrently in an operating system**. This allows a specific program to handle many user requests at the same time.**

- **Communication can be related or unrelated (**using pipes one process running in one terminal and another process in another terminal**) processes**
- **Linux supports a number of inter-process communication** (IPC)
  - Signals, shared memory and pipes are the most common
  - **But other IPCs are**
    -shared files
    -shared memory with (with semaphores)
    -message Queues
    -sockets

Inter-process communication Approaches

a) **Pipes**
b) **Message passing**
c) **Message queues**
d) **Direct communication**

e) **Indirect communication**
f) **Shared memory**
g) **FiFo**

Pipes

Pipes is widely used for communication between two related processes. This is a half-duplex method, so the first process communicates with the second process. However, in order to achieve a full-duplex, another pipe is needed.

Message passing

The process communicates with each other without resorting to shared variables.

Message Queues

A message queue is a linked list of messages stored within the kernel. It is identified by a message queue identifier. This method offers communication between single or multiple processes with full-duplex capacity.

Direct communication

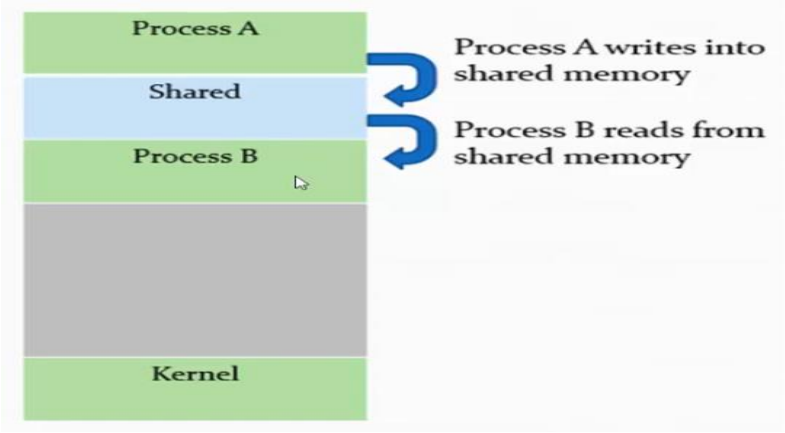
In this method, a link is established one pair of communicating process and between each pair, only one link exists.

What is Shared memory?

Shared memory is the fastest interposes communication mechanism. The operating systems maps a memory segment in the address space of several processes. **So that several processes can read and write in that memory segment without calling operating system** functions. However, we need some kind of between processes that read and write shared memory.

Why use processes use shared memory?

- Shared memory is memory that may be simultaneously accessed multiple programs with an intent to provide communication among them or avoid redundant copies.
- Shared memory run on single processor, parallel processor, or clustered processors



Shared memory is a concept where two or more process

Share resources from one memory address space.

Example:

Consider what happens when a server process wants to send an HTML file to a client process that resides in the same machine using network mechanisms.

- Single server that enables clients and servers to exchange data and instructions using main memory.
- Without shared memory, both client and server use use operating system buffers to accomplish the modification and exchange.
- With shared memory, the client writes its process directly into RAM and issues a semaphore value to flag sever attention. There is only 1 read and 1 write per communication, and the read/write is considerably faster than using system services. – to achieve higher performance computing

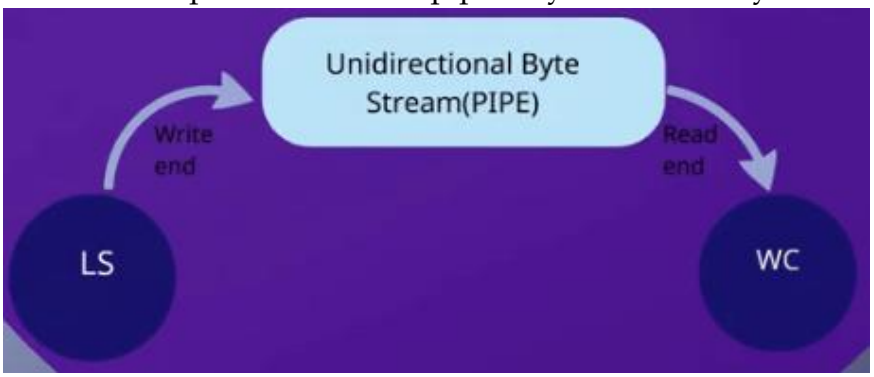
- **Need of inter process communication**
  i. Information sharing
  ii. Speedup
  iii. Modularity
  iv. Convenience

System V IPC is more heavy weight than BSD mmap and provides three methods of communication.

- Message Queues
- Semaphore and
- Shared segment.

Pipes and pipe demonstration using P open () and P close () functions

- Pipe is unidirectional (moving in a single direction) Byte stream. It has two ends means that two processes are connected to each other. (**connection between two processes**)
- Unlike other processes in **IPC** pipe only offer one-way communication.



LS– list the content of the directory

WC- word count the number of lines, bytes, words output from first command to input of second, and so forth

- Pipe () is a Linux system function.
- The pipe () system function is used to open **file descriptors** which are used to communicate between different Linux processes.
- Pipe is a special file that follows the FIFO mechanism

All about pipe () function:

The syntax of the pipe () function is:

int pipe (int pipefd [2]);

here, the pipe () function creates a unidirectional data channel for inter-process communication. You pass in an int(integer) type array pipefd consisting of 2 array element to the function pipe () then the pipe () function creates two file descriptors in the pipefd array.

The first element of the pipefd array, pipefd [0] is used for reading data from the pipe.

The second element of the pipefd array, pipefd [1] is used for writing data to the pipe.

On success, the pipe () function return 0. If an error occurs during pipe initialization, then pipe returns -1

Why do we use/Benefits of using pipes?

1. You can use them across different systems that may not exist at the same time. Suppose you want real-time communication of different machines. Just use FIFO files to share data.
2. As a file you can control ownership and permissions
3. You don't have to start the reading/ writing processes at the same time.

Example 1;

To create a new C source file pipe.c

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>

int main (void)
{
int pipefds[2];
if(pipe(pipefds)== -1)
{perror("pipe");
exit(EXIT_fAILURE);
}
printf("Read file Descriptor value:%d\n",pipefds[0]);
printf("write file Descriptor value:%d\n",pipefds[1];
return EXIT_SUCCESS;
}
```

Codes explained

Exit (EXIT_FAILURE)

If code fails the it exits there.

Popen and Pclose functions for inter-process communication.

Popen – opens a pipe stream to or from a process

- A common use of pipes is to send data to or receive data from a program being run in a sub process.
- **Popen** and **Pclose** functions ease this paradigm by eliminating the need to invoke pipe, fork, dup2, exec, and fd open.

Syntax

```
#include<stdio.h>
FILE  *popen(const char *command, const char *type);
```

Process for communication

send(or receive data

How to access the Linux manual

```
:~$ man open
```

```
POPEN(3)                    Linux Programmer's Manual                    POPEN(3)

NAME
       popen, pclose - pipe stream to or from a process

SYNOPSIS
       #include <stdio.h>

       FILE *popen(const char *command, const char *type);

       int pclose(FILE *stream);

   Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

       popen(), pclose():
           _POSIX_C_SOURCE >= 2
               || /* Glibc versions <= 2.19: */ _BSD_SOURCE || _SVID_SOURCE

DESCRIPTION
       The  popen()  function  opens  a process by creating a pipe, forking, and invoking the shell.
       Since a pipe is by definition unidirectional, the type argument may specify only  reading  or
       writing, not both; the resulting stream is correspondingly read-only or write-only.

       The  command  argument  is  a  pointer to a null-terminated string containing a shell command
Manual page popen(3) line 1 (press h for help or q to quit)
```

Program to write into a pipe

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<string.h>
int main()
{
FILE *rd;
char buffer[50];
sprintf(buffer,"name first");
rd=popen("wc -c","w");
fwrite(buffer,sizeof(char),strlen(buffer),rd);
pclose(rd);
}
```

File* rd;

Data type file stream= file * rd the file name.

Char buffer [50] = buffer as a stream storage.

Sprintf () = print string to write data into the buffer

rd=popen ("wc –c", "w"); = rd name of the file that will use open pipe using write mode.

fwrite(buffer,sizeof(char),strlen(buffer),rd;

wc = cmd that reads either standard input or a list of computer files and generate one of the following statistics newline, word count, and byte count.

Example to illustrate the above written codes

```
 GNU nano 4.8                                    ipc1.c
#include<stdio.h>
#include<string.h>
int main()
{
        FILE *rd;
        char buffer[50];
        sprintf(buffer, "name first");
        rd=popen("wc -c", "w");
        fwrite(buffer,sizeof(char), strlen(buffer), rd);
        pclose(rd);
}
```

```
baljit@DESKTOP-MMRRHBE:~$ nano ipc1.c
baljit@DESKTOP-MMRRHBE:~$ gcc ipc1.c
baljit@DESKTOP-MMRRHBE:~$ ./a.out
10
baljit@DESKTOP-MMRRHBE:~$
```

UNIX Thread management and synchronization

Posix standards and its sub group

- **What is posix standards? "**Portability operating system interface**"** is the name of a family related standards specified by IEEE to define the application programming interface (API), along with shell utilities interface for the software compatible with variants of the UNIX operating system.

- Because many versions of UNIX exist today and each of them provides its own set of API functions, it is difficult for system developers to create applications that can be easily ported to different versions of UNIX.

- Some of **the subgroups** of POSIX are POSIX.1, POSIX.1b & POSIX.1c are concerned with the development of set of standards for system developers.

1. **POSIX. 1**
   - This committee proposes a standard for a base operating system API; this standard specifies APIs for the manipulating of files and processes.
   - It is formally known as IEEE standard 1003.1- 1990 and it was also adopted by the ISO as the international standard ISO/IEC 9945: 1: 1990.

2. **POSIX.1b**
   - This committee proposes a set of standard APIs for a real time OS interface; these include IPC (interposes communication)
   - This standard is formally known as IEEE standard 10034-1993.

3. **Posix.1c**
   - This standard specifies multi-threaded programming interface. This is the newest POSIX standard.

- These standards are proposed for a generic OS that is not necessary be UNIX system.
- E.g VMS from Digital Equipment corporation, OS/2 from IBM, & Windows NT from Microsoft corporation are POSIX-compliant, yet they are not UNIX systems.
- To ensure a user program conforms to POSIX.1 standard the user should either define the manifested constant_ POSIX_SOURCE at the beginning of each source module of the program (before inclusion of any header) as; # define_POSIX_SOURCE OR specify the –D_ POSIX_SOURCE option to a C++ compiler(CC) in a compilation; % CC –D_ POSIX_SOURCE *.C
- POSIX.1b defines different manifested constant to check conformance of user program to that standard. The new macro is POSIX_C_SOURCE and its value indicates POSIX version to which a user program conforms. Its value can be:

- **Program to check and display _POSIX_VERSION constant of the system on which it is run**

```
#define  _POSIX_SOURCE
#define  _POSIX_C_SOURCE   199309L
#include<iostream.h>
#include<unistd.h>
int main()
{
#ifdef _POSIX_VERSION
      cout<<"System conforms to POSIX"<<"_POSIX_VERSION<<endl;
#else
      cout<<"_POSIX_VERSION undefined\n";
#endif
      return 0;
}
```

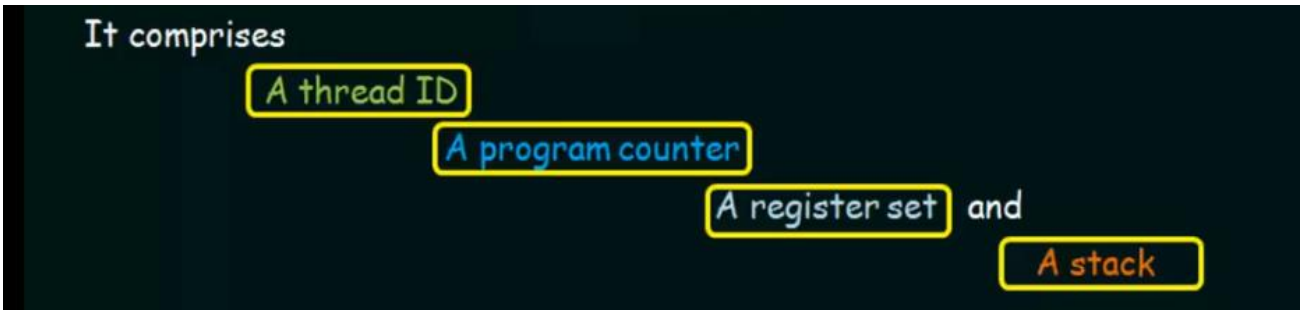
Threading Concept in systems programming

Knowing how to properly use threads should be part of every computer science and engineering student repertoire.

Threads?

Def 1: A thread is a basic unit of CPU utilization.

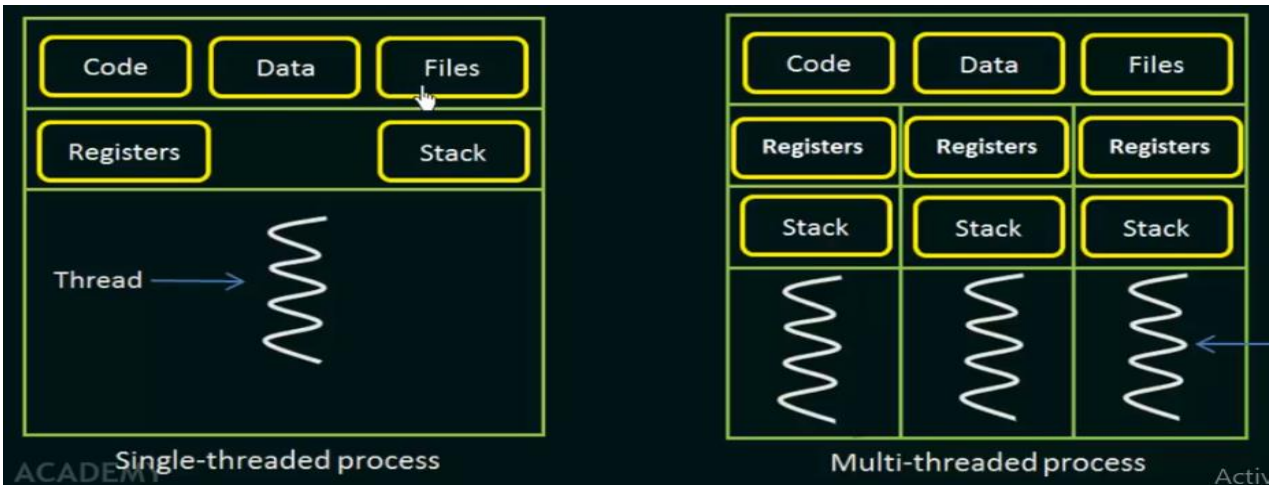
Def 2: A thread of execution is often regarded as the smallest unit of processing that a scheduler works on.



i. **A program counter-** is a register that manages the memory address of the instruction to be executed next…
ii. **A register set -** processor can be partitioned into different register classes.
iii. **A stack –** a mount of memory allocated to the stack in each thread.

It shares with other threads belonging to the same process its code section, data section, and other operating- system resources, such as open files and signals.

A traditional / heavyweight process has a single thread of control. If a process has multiple threads of control, it can perform more than one task at a time.



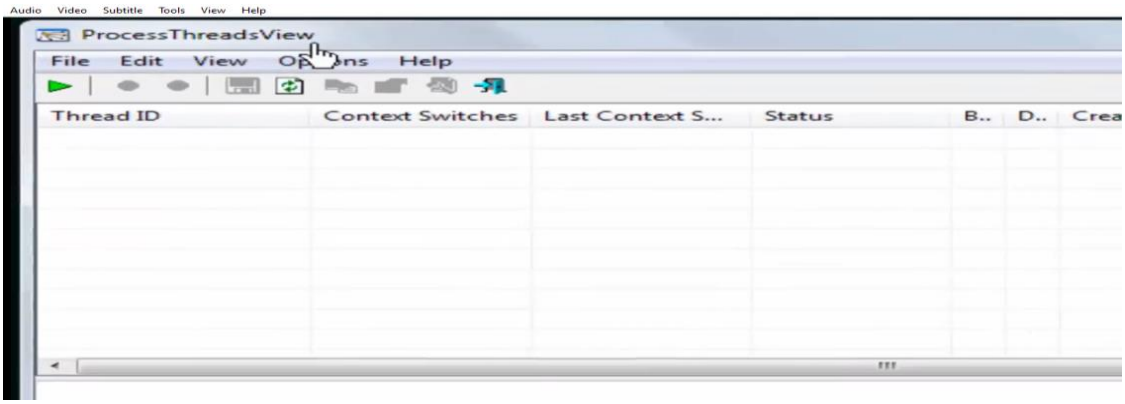


Fig 1 : process threads view

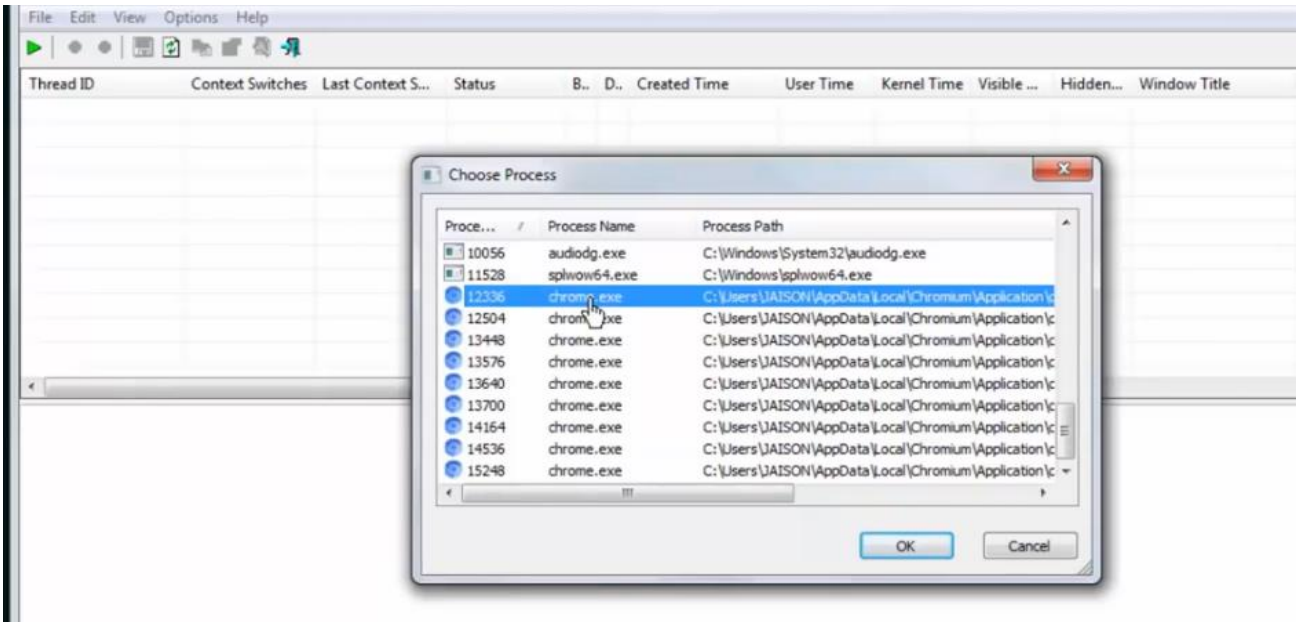


Fig 2: showing the different processes

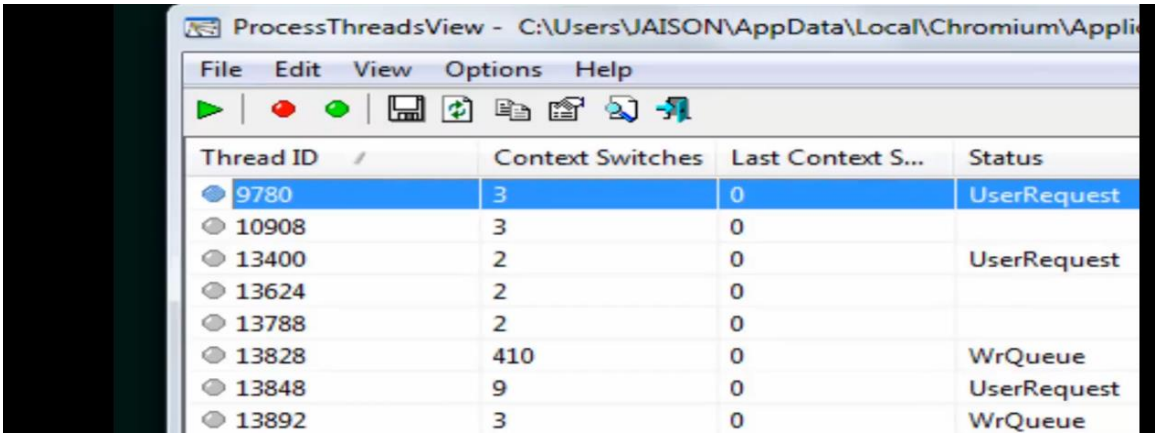
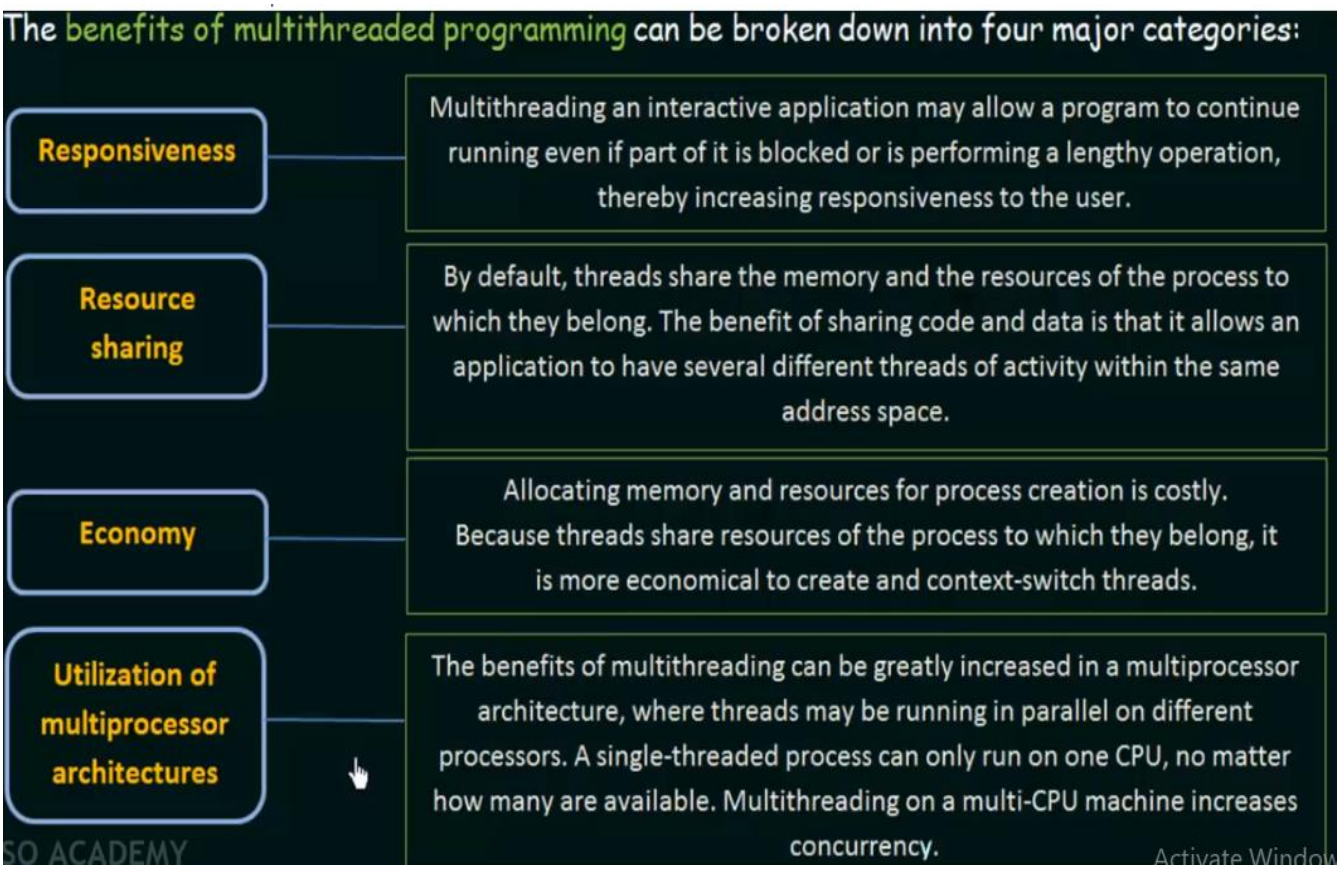


Fig 3: details concerning the threads from processor illustrated in Fig 2.



Threads are useful – Example applications

Example 1: A web serer

- It needs to handle several download requests over a short period
- Hence more efficient to create (and destroy) a single thread for each request
- Multiple threads can possibly be executing simultaneously on different processors

Example 2: A responsive user interface

- One area in which threads can be very helpful is in user-interface programs. These programs are usually centered around a loop of reading user input, processing it, and showing the results of the processing. The processing part may sometimes take a while to complete, and the user is made to wait during this operation. By placing such long operations in a separate thread to read, while having another thread to read user input, the program can be more responsive. It may allow the user to cancel the operation in the middle.

Thread Identification

Just as a process is identified through a **process ID**, a thread is identified by a **thread ID**. But interestingly, the similarity between the two ends here.

A **process ID** is unique across the system where as a thread ID is unique only in context of a single process.

A **process ID is an** integer value but the thread ID is not necessarily an integer value. It could well be a structure

A **process ID can be printed very easily** while a thread ID is not easy to print.

The above points give an idea about the difference between a process ID and thread ID.

Thread Creation

Normally when a program starts up and becomes a process, it starts with a default thread. So we can say that every process has at least one thread of control. **Thread ID** is represented by the type 'pthread_t'.

What are pthreads?

Historically, hardware vendors have implemented their own proprietary versions of threads. These implementations differed substantially from each other, making it difficult for programmers to develop portable threaded applications.

In order to take full advantage of the capabilities provided by threads, a standardized programming interface was required. for UNIX systems, **this interface has been specified by the IEEE POSIX 1003.1c standard (1995).** implementations which adhere to this standard are referred to as POSIX threads or Pthreads. Most hardware vendors now offer Pthreads in additional to their proprietary threads.

How to Create Threads ID in Linux (With a C Example Program)

We must include the pthread.h header file at the beginning of the script to use all the functions of the pthreads library.

How do I find thread ID?

The pthread_self () function is used to get the ID of the current thread. But if there are multiple threads, and one thread is completed, then that ID can be reused. Because it remains unchanged during its lifetime.

```
#include<stdio.h>
#include<stlib.h>
#include<pthread.h>
void* func(void*p)
{
printf("from the function,the thread id = %d\n"",pthread_self()); // get current thread id
pthread_exit(NULL);
return Null;
}
main() {
pthread_t thread; // declare thread
pthread_create (&thread,NULL,func,NULL);
print("from the main function,the thread id=%d\n", thread");
pthread_join(thread,NULL);// join with the main thread
}
```

Output

In function

Thread id =1

In main

Thread id =1

Void* func (void*p) – pointer which is not associated with any data types. It's a general purpose pointer. means function does not pass any argument.

Pthread_create (&thread, NULL, func, NULL) = creates a new thread in the calling process, if attr is NULL the default attributes shall be used.

Pthread_join – this is useful when you want to wait for thread/s to terminate before further processing in main thread.

Mutex and semaphore in systems administration

- **Read on mutex vs semaphore**

Javatpoint search mutex and semaphore

www.Guru99.com

- **when should we use semaphores and when should we use mutex?**
- **Read semaphore and mutex implementation using c programming**